

Pipeline de Dados com PySpark, dbt, Airflow e Databricks sobre Microdados Públicos do Novo Bolsa Família

Arquitetura Medallion (Bronze · Silver · Gold) com Delta Lake, transformação em dbt, orquestração em Airflow, testes automatizados e CI/CD

Autora: Jessica Van Klaveren

Objetivo: Projeto de portfólio para posições de Engenharia de Dados Nível Pleno

Fonte de dados: Portal da Transparência (CGU) — dados abertos do Governo Federal

Status deste documento: Escopo fechado em 15/09/2026 — pré-implementação, código a ser desenvolvido nas fases descritas na Seção 15. Nenhuma nova competência entra no escopo depois deste ponto; ajustes de execução (ex.: quais meses da amostra histórica) podem ocorrer dentro do já definido.

Escopo final: PySpark (bronze/silver) + Delta Lake + dbt (gold) + Airflow (orquestração) + Databricks Community Edition + LGPD/pseudonimização + amostra histórica multi-ano (mar/2023–atual) + comparação VAR vs. rede neural + publicação em GitHub/dbt docs/README

Data: Setembro de 2026

Sumário

1. Sumário executivo
2. Contexto e motivação
3. Objetivos e critérios de sucesso
4. Fonte de dados: Novo Bolsa Família
5. Privacidade, LGPD e governança de dados
6. Arquitetura da solução (Medallion + Orquestração + Transformação)
7. Design técnico das camadas Bronze e Silver (PySpark)
8. Transformação analítica com dbt (Silver → Gold)
9. Orquestração com Airflow
10. Amostra histórica e comparação de modelos de previsão (VAR vs. rede neural)
11. Publicação e visibilidade para recrutadores
12. Qualidade de dados e testes
13. Stack tecnológico e justificativa
14. Estrutura do repositório
15. Plano de execução por fases
16. Riscos, limitações e decisões em aberto
17. Conexão com o mercado de trabalho
18. Referências

1. Sumário executivo

Este documento especifica um projeto de engenharia de dados de ponta a ponta, construído sobre um dataset público real e volumoso do Governo Federal brasileiro (pagamentos do programa Novo Bolsa Família), com o objetivo explícito de demonstrar, de forma verificável, competências hoje ausentes do portfólio da autora e recorrentemente exigidas em vagas de Engenheiro(a) de Dados Pleno/Sênior no mercado brasileiro: **Apache Spark (PySpark), arquitetura em camadas (medallion: bronze/silver/gold), Delta Lake, transformação analítica em dbt, orquestração com Airflow** e **Databricks**. A decisão de cobrir todo esse conjunto em um único projeto (em vez de dividir em entregas menores) foi tomada conscientemente pela autora em 15/09/2026, com o trade-off de escopo/risco explicado na Seção 16.

O escopo foi validado com dados reais antes da escrita deste documento: o arquivo mensal de pagamentos (competência 2026-01) foi baixado diretamente do Portal da Transparência, totalizando **19.480.035 linhas** e **~2,15 GB descompactados** em um único mês — volume que justifica genuinamente processamento distribuído em vez de pandas puro em memória.

Compromisso deste documento

Nenhum dado, coluna, volume ou comportamento descrito aqui foi assumido ou estimado sem conferência direta no arquivo real. Onde uma decisão ainda depende de teste empírico (ex.: tempo de execução, comportamento no Databricks Community Edition), isso está marcado explicitamente como pendente na Seção 12.

2. Contexto e motivação

Durante a preparação do currículo da autora para vagas de Engenharia de Dados Pleno, uma pesquisa de mercado em vagas ativas (Gupy, LinkedIn, Indeed, GeekHunter, setembro/2026) identificou um padrão consistente: a maioria das vagas Pleno/Sênior exige, além de Python/SQL/APIs (já dominados), pelo menos dois dos seguintes itens — **PySpark/Spark, Databricks, orquestração (Airflow), transformação (dbt)** e **arquitetura lakehouse/medallion**. Esse projeto foi desenhado para fechar a lacuna de Spark/Databricks/medallion com evidência real e revisável em repositório público, e não apenas com a palavra-chave no currículo.

3. Objetivos e critérios de sucesso

3.1 Objetivo primário

Construir e publicar um pipeline de dados completo, funcional e testado, que ingere e limpa os microdados do Novo Bolsa Família em PySpark, transforma e agrega via dbt, é orquestrado por Airflow, e roda tanto localmente quanto no Databricks Community Edition.

3.2 Definição de "pronto" (Definition of Done)

- ☐ Pipeline com as três camadas (bronze/silver/gold) implementado e executável localmente com um único comando
- ☐ Bronze e Silver implementados em PySpark; Silver → Gold implementado como modelos dbt (dbt-databricks)
- ☐ Dados persistidos em formato Delta Lake em cada camada, com histórico de versões (time travel) demonstrado
- ☐ Camada Silver com pseudonimização de NIS e nome do beneficiário (ver Seção 5)
- ☐ DAG do Airflow orquestrando bronze → silver → dbt run → testes de qualidade, com agendamento mensal simulado
- ☐ Testes dbt (schema tests: not_null, unique, accepted_values, relationships) passando em todos os modelos Gold
- ☐ Suíte de testes automatizados (pytest) cobrindo as transformações PySpark críticas, rodando em CI (GitHub Actions) a cada push
- ☐ Validações de qualidade de dados explícitas (schema, nulos, faixas de valor) com relatório de execução
- ☐ Notebook/job equivalente rodando com sucesso no Databricks Community Edition, com print/export do resultado como evidência
- ☐ README com arquitetura, decisões de design e instruções de reprodução
- ☐ Repositório público no GitHub, linkado no currículo

Regra explícita contra projeto pela metade

Nenhuma etapa da Seção 15 é considerada concluída sem o item correspondente desta lista marcado E verificado (código rodando, teste passando, evidência visual). "Escrevi o código mas não testei" não conta como fase concluída.

4. Fonte de dados: Novo Bolsa Família

Dataset oficial de dados abertos do Governo Federal, disponibilizado pela Controladoria-Geral da União (CGU) através do Portal da Transparência, sem necessidade de cadastro ou autenticação.

Item	Valor confirmado
URL de acesso	portaldatransparencia.gov.br/download-de-dados/novo-bolsa-familia/{AAAAAMM}
Redireciona para	dadosabertos-download.cgu.gov.br/PortalDaTransparencia/saida/novo-bolsa-familia/
Granularidade	Uma parcela paga a um beneficiário, por mês de competência
Arquivo testado	Competência 202601 (janeiro/2026)
Linhas confirmadas	19.480.035

Tamanho descompactado	2.151.373.680 bytes (~2,15 GB)
Tamanho compactado (.zip)	346.395.830 bytes (~346 MB)
Encoding	ISO-8859-1 (Latin-1) — requer conversão explícita para UTF-8
Delimitador	; (ponto e vírgula), campos entre aspas duplas
Quebra de linha	CRLF (Windows)
Formato numérico	Vírgula decimal (ex.: "850,00") — requer parse manual, não é float direto
Cobertura geográfica	27 UFs confirmadas em amostra de 500.000 linhas (26 estados + DF)

4.1 Dicionário de dados (colunas reais do arquivo)

Coluna (original)	Tipo bruto	Descrição	Sensibilidade
MÊS_COMPETÊNCIA	string (AAAAMM)	Mês de referência do arquivo/pagamento	Baixa
MÊS_REFERÊNCIA	string (AAAAMM)	Mês a que a parcela se refere (pode diferir da competência por pagamentos retroativos)	Baixa
UF	string (2 letras)	Unidade federativa do beneficiário	Baixa
CÓDIGO_MUNICÍPIO_SIAFI	string numérica	Código SIAFI do município	Baixa
NOME_MUNICÍPIO	string	Nome do município	Baixa
CPF_FAVORECIDO	string mascarada	CPF já parcialmente mascarado na fonte (ex.: ***.582.082-**); frequentemente vazio	Média (já mascarado)
NIS_FAVORECIDO	string numérica	Número de Identificação Social — identificador pessoal único, não mascarado	Alta
NOME_FAVORECIDO	string	Nome completo do beneficiário — não mascarado	Alta
VALOR_PARCELA	string (vírgula decimal)	Valor pago na parcela, em reais	Baixa

Amostra extraída diretamente do arquivo com Python/csv em 15/09/2026, encoding ISO-8859-1. As três primeiras linhas reais confirmam o formato acima; nenhuma coluna foi inferida sem leitura direta.

5. Privacidade, LGPD e governança de dados

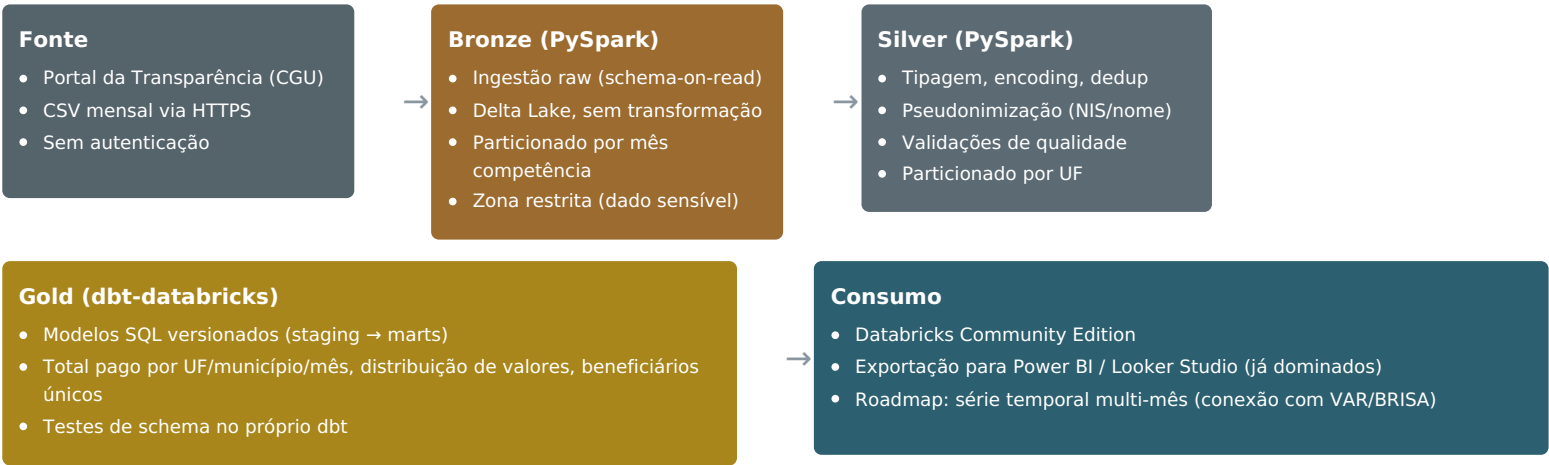
Embora o dado seja público e legalmente disponibilizado pelo governo sob a Lei de Acesso à Informação, ele contém **NIS** e **nome completo** não mascarados — dados pessoais na definição da LGPD (Lei 13.709/2018). Um projeto de engenharia de dados de nível sênior precisa tratar isso como questão de design, não como detalhe a ignorar.

5.1 Medidas adotadas no design

- **Camada Bronze:** mantém o dado bruto como veio da fonte (necessário para auditoria/reprocessamento), mas é tratada como zona restrita — não exposta em dashboards nem em nenhuma camada de consumo.
- **Camada Silver em diante:** `NIS FAVORECIDO` e `NOME FAVORECIDO` são pseudonimizados via hash (SHA-256 com salt) antes de qualquer agregação ou análise. O nome completo não é propagado para a camada Gold em nenhuma hipótese — as agregações de negócio (Seção 7.3) não precisam dele.
- **Documentação explícita** no README do repositório sobre a natureza do dado, a finalidade exclusivamente educacional/portfólio, e que o repositório não deve ser usado para reidentificação de beneficiários.

6. Arquitetura da solução (Medallion + Orquestração + Transformação)

Arquitetura em três camadas (padrão "lakehouse/medallion architecture", citado na documentação oficial do Databricks e nas vagas pesquisadas), com PySpark responsável pela ingestão/limpeza (Bronze/Silver), dbt responsável pela transformação analítica (Silver → Gold) e Airflow orquestrando o ciclo completo.



Camada de orquestração (Airflow) — cobre o fluxo inteiro

Uma DAG única aciona, em sequência e com dependência explícita entre tasks: (1) ingestão Bronze em PySpark → (2) limpeza/pseudonimização Silver em PySpark → (3) `dbt run` (Silver → Gold) → (4) `dbt test` (validação dos modelos Gold) → (5) notificação de sucesso/falha. Detalhes na Seção 9.

7. Design técnico das camadas Bronze e Silver (PySpark)

7.1 Bronze

Leitura do CSV original com `spark.read.csv`, opções `sep=";"`, `encoding="ISO-8859-1"`, `header=True`, todas as colunas mantidas como `string` (schema-on-read — evita falha de ingestão por mudança de formato na fonte). Gravação em Delta Lake particionado por `mes_competencia`. Nenhuma linha é descartada nesta camada — é a fonte da verdade para reprocessamento.

7.2 Silver

Transformação	Detalhe
Conversão de encoding	ISO-8859-1 → UTF-8 (acentuação de nomes de município)
Tipagem	<code>mes_competencia / mes_referencia</code> → <code>date</code> (primeiro dia do mês); <code>valor_parcela</code> : troca de vírgula por ponto e cast para <code>decimal(10,2)</code>
Pseudonimização	<code>nis_hash = sha2(concat(nis, salt), 256)</code> ; coluna <code>nome_favorecido</code> removida desta camada em diante
Deduplicação	<code>dropDuplicates</code> por chave composta (<code>nis_hash</code> , <code>mes_referencia</code> , <code>valor_parcela</code>)
Tratamento de nulos	CPF vazio → mantido nulo (esperado, não é erro); linhas sem NIS → quarentena (gravadas separadamente para investigação, não descartadas silenciosamente)
Particionamento	Por <code>uf</code>

7.3 Entrega da camada Silver para o dbt

A responsabilidade do PySpark termina na Silver. A camada Gold **não** é gerada em PySpark neste projeto — é gerada por modelos dbt lendo diretamente as tabelas Delta da Silver, conforme detalhado na Seção 8. Essa divisão é proposital: mantém o PySpark focado no que ele faz melhor (processamento de volume bruto, limpeza, dados semi-estruturados) e o dbt focado no que ele faz melhor (transformação analítica em SQL, testada e documentada).

8. Transformação analítica com dbt (Silver → Gold)

O dbt lê as tabelas Delta da camada Silver (via adapter `dbt-databricks`, que fala nativamente com tabelas Delta/Unity Catalog) e materializa os modelos Gold como tabelas SQL versionadas, testadas e documentadas — o padrão de mercado citado nas vagas pesquisadas como "SQL avançado + dbt".

8.1 Estrutura de modelos (padrão staging → marts)

Camada dbt	Modelo	Responsabilidade
<code>staging</code>	<code>stg_pagamentos.sql</code>	Seleção/renomeação 1:1 da tabela Silver, sem lógica de negócio (padrão dbt para isolar a fonte)
<code>marts</code>	<code>gold_pagamentos_uf_mes.sql</code>	Soma/média de <code>valor_parcela</code> e contagem de beneficiários únicos, por UF e mês
<code>marts</code>	<code>gold_pagamentos_municipio_mes.sql</code>	Mesma agregação em nível de município
<code>marts</code>	<code>gold_distribuicao_valores.sql</code>	Histograma de faixas de valor pago (detecção de concentração/outliers)

8.2 Testes dbt (declarativos, em `schema.yml`)

- `not_null` e `unique` nas chaves de agregação (`uf` + `mes`, `municipio` + `mes`)
- `accepted_values` na coluna UF (as 27 siglas válidas)
- `relationships` : todo município em `gold_pagamentos_municipio_mes` deve existir na dimensão de municípios da Silver (teste de integridade referencial)
- Teste customizado: soma total de `valor_parcela` na Gold == soma direta na Silver (mesma verificação de consistência ponta a ponta já prevista no plano de fases)

8.3 Documentação

`dbt docs generate` produz um site navegável com a linhagem (lineage graph) `staging` → `marts`, descrição de cada coluna e teste associado — evidência visual de que o pipeline analítico é compreensível por terceiros, não uma caixa-preta.

9. Orquestração com Airflow

Uma DAG única representa o pipeline completo, com cada camada como uma task e dependências explícitas entre elas.

9.1 Desenho da DAG





9.2 Configurações relevantes

Aspecto	Decisão
Agendamento	<code>schedule_interval</code> mensal, simulando a chegada de um novo arquivo de competência todo mês
Retries	2 tentativas com backoff exponencial por task, antes de marcar falha
Idempotência	Cada task pode rodar de novo sem duplicar dado (overwrite por partição na Bronze/Silver, <code>--full-refresh</code> controlado no dbt)
Ambiente de execução local	Airflow via <code>docker-compose</code> (imagem oficial <code>apache/airflow</code>) — não requer infraestrutura paga
Integração com dbt	Avaliar <code>astronomer-cosmos</code> (provider que converte um projeto dbt inteiro em tasks nativas do Airflow) como alternativa ao BashOperator simples — decisão tomada na fase correspondente (Seção 15)

Ponto de honestidade técnica

Airflow local via Docker Compose demonstra domínio da ferramenta (DAGs, operators, scheduling, retries), mas não é o mesmo que operar Airflow gerenciado (MWAA, Cloud Composer, Astronomer) em produção com múltiplos times. Isso será declarado explicitamente no README, sem sugerir experiência que não existe.

10. Amostra histórica e comparação de modelos de previsão (VAR vs. rede neural)

Extensão da camada Gold para responder a uma pergunta de negócio direta: o valor total pago pelo Novo Bolsa Família está subindo ou caindo ao longo do tempo? Esta seção só entrou no escopo em 15/09/2026, por decisão explícita da autora, e usa a mesma disciplina de validação empírica das demais.

10.1 Amostra histórica

Confirmado por acesso direto: a fonte tem dado real disponível desde a competência 202303 (março/2023, relançamento do programa como "Novo Bolsa Família") até a competência mais recente. Baixar as ~35 competências completas exigiria ~70 GB, inviável neste ambiente. Decisão de engenharia: amostragem de **um mês por trimestre** (~12 competências entre mar/2023 e hoje), processada uma competência por vez (ingestão → Delta → descarte do CSV bruto) para manter o uso de disco sob controle. Cada competência amostrada passa pelo mesmo pipeline Bronze/Silver já especificado — não é um caminho paralelo.

10.2 Novo modelo Gold

`gold_evolucao_temporal_nacional` (modelo dbt adicional): valor total pago e número de beneficiários únicos, por mês amostrado, em nível nacional e por UF — a série temporal que alimenta a comparação de modelos.

10.3 Comparação de modelos de previsão

Modelo	Abordagem	Reaproveitamento
VAR (Vector Autoregression)	<code>statsmodels.tsa.api.VAR</code> , mesma metodologia de seleção de lag (AIC/BIC) e backtesting (RMSE/MAE/MAPE) já usada no projeto Pluviômetro (BRISA)	Direto — mesmo código-base, adaptado para a série de pagamentos
Rede neural simples (MLP ou LSTM)	Keras/TensorFlow, rede rasa (1-2 camadas ocultas) sobre a mesma série — escopo deliberadamente simples, não é o objetivo competir em sofisticação, é demonstrar aplicação real do conteúdo do MBA em Ciência de Dados	Novo — primeira aplicação prática documentada desta competência

Critério de comparação: mesmo split treino/teste, mesmas métricas (RMSE/MAE/MAPE) para os dois modelos, resultado reportado lado a lado — sem alegar qual "é melhor" sem o número que sustente a afirmação.

Limitação honesta desta seção

~12 pontos mensais é uma série curta para deep learning (que tipicamente precisa de mais dados para superar métodos clássicos). É esperado, e será reportado com transparência, que o VAR performe igual ou melhor que a rede neural nessa escala — o valor da comparação está em demonstrar competência técnica com ambas as abordagens e saber interpretar quando cada uma se aplica, não em "vencer" com a rede neural.

11. Publicação e visibilidade para recrutadores

- Repositório público no GitHub** (`github.com/jklaveren`), fixado ("pinned") no perfil ao final do projeto.
- README como vitrine principal** — a maioria dos recrutadores lê o README em vez de rodar o código: resumo de uma frase, diagrama

- de arquitetura, print/GIF da DAG do Airflow rodando, print do site do dbt docs, gráfico da comparação VAR vs. rede neural, instruções de reprodução.
- **dbt docs hospedado via GitHub Pages**, gratuito, linkado no README.
 - **Link no currículo**, mesmo padrão já usado para WhoDados e o projeto de scraping.
 - **Evidências coletadas fase a fase** (não só no final) — cada fase da Seção 15 que gerar uma saída visível (print, gráfico, log de execução) salva essa evidência imediatamente, para que o projeto seja apresentável mesmo se parar antes do fim.

12. Qualidade de dados e testes

12.1 Validações automáticas (bronze → silver, em PySpark)

- Contagem de linhas: `silver_count + quarentena_count == bronze_count` (nenhuma linha perdida silenciosamente)
- Schema enforcement: falha explícita se uma coluna esperada não existir no arquivo de origem
- `valor_parcela` dentro de faixa plausível (`> 0` e `<` valor máximo documentado do programa)
- UF pertence ao conjunto válido de 27 siglas

12.2 Validações declarativas (silver → gold, em dbt)

Ver Seção 8.2 — testes dbt cobrem a camada de transformação analítica.

12.3 Testes automatizados (PySpark)

Suíte `pytest` com `pyspark.testing` (ou `chispa`) para testar funções de transformação isoladamente, com DataFrames pequenos e controlados (não o dataset completo) — mesma prática já usada no projeto Pluviômetro (backtesting do modelo VAR). Execução automática via GitHub Actions a cada push, reaproveitando o padrão de CI/CD já usado no projeto WhoDados.

13. Stack tecnológico e justificativa

Componente	Ferramenta	Justificativa
Processamento distribuído	PySpark (local standalone)	Volume real de 19,5M linhas/mês inviabiliza pandas puro em memória padrão
Formato de armazenamento	Delta Lake (pacote <code>delta-spark</code> , open source, sem depender do Databricks)	ACID, time travel, schema evolution — padrão citado nas vagas pesquisadas
Transformação analítica	dbt (adapter <code>dbt-databricks</code>)	Padrão de mercado citado junto de SQL avançado nas vagas pesquisadas; roda direto sobre Delta/Unity Catalog
Orquestração	Apache Airflow (local, via Docker Compose)	Ferramenta de orquestração mais citada nas vagas Pleno/Sênior pesquisadas
Ambiente gerenciado	Databricks Community Edition	Grátis, permite falar "Databricks" com evidência real de uso, não só teoria
Testes	pytest (PySpark) + testes nativos do dbt (Gold)	pytest já dominado (usado no WhoDados); testes dbt cobrem a camada SQL
CI/CD	GitHub Actions	Já dominado (usado no WhoDados e Pluviômetro)
Controle de versão	Git / GitHub público	Verificabilidade do trabalho por recrutadores

14. Estrutura do repositório (proposta)

```
bolsa-familia-pipeline/
├── README.md                # arquitetura, decisões, como reproduzir
├── requirements.txt
├── docker-compose.yml       # Airflow local (webserver, scheduler, postgres)
├── data/
│   └── raw/                 # .gitignore — dado bruto não versionado
├── src/
│   ├── bronze/
│   │   └── ingest.py
│   ├── silver/
│   │   ├── clean.py
│   │   └── pseudonymize.py
│   ├── quality/
│   │   └── checks.py
├── dbt/
│   └── bolsa_familia/
│       ├── dbt_project.yml
│       ├── models/
│       │   ├── staging/
│       │   │   ├── stg_pagamentos.sql
│       │   │   └── sources.yml
│       │   └── marts/
│       │       ├── gold_pagamentos_uf_mes.sql
│       │       ├── gold_pagamentos_municipio_mes.sql
│       │       ├── gold_distribuicao_valores.sql
│       │       └── schema.yml # testes declarativos (Seção 8.2)
│       └── tests/           # testes customizados (singular tests)
└── dags/
```

```
|└─ bolsa_familia_pipeline_dag.py
└─ notebooks/
  └─ databricks_pipeline.py    # versão para Databricks Community Edition
└─ tests/
  └─ test_silver_clean.py
  └─ test_pseudonymize.py
└─ .github/
  └─ workflows/
    └─ ci.yml                  # pytest + dbt test em cada push
```

15. Plano de execução por fases

Fase	Entregável	Critério de conclusão (verificável)
0 — Validação de escopo	Este documento	Concluída: dado real baixado e inspecionado (19.480.035 linhas confirmadas)
1 — Ambiente	PySpark + delta-spark instalados e testados localmente	Script mínimo lê um DataFrame e grava em Delta com sucesso
2 — Bronze	Ingestão do arquivo real (202601) para Delta	Contagem de linhas na tabela Bronze == 19.480.035
3 — Silver	Limpeza, tipagem e pseudonimização	Testes pytest passando; nome_favorecido ausente da tabela; contagem bate com bronze – quarentena
4 — Setup dbt	Projeto dbt inicializado, conectado à Silver	<code>dbt debug</code> conecta com sucesso; <code>stg_pagamentos</code> materializa e bate em contagem com a Silver
5 — Modelos Gold em dbt	3 modelos marts + testes de schema	<code>dbt run</code> e <code>dbt test</code> passam 100%; soma da Gold bate com a Silver
6 — Qualidade e CI (PySpark)	Checks automáticos + GitHub Actions	Pipeline de CI verde no repositório público
7 — Airflow local	DAG completa rodando via Docker Compose	Execução manual da DAG do início ao fim sem falha, com as 5 tasks da Seção 9.1 verdes na UI do Airflow
8 — CI estendido	<code>dbt test</code> incluído no GitHub Actions	CI falha de propósito se um teste dbt for quebrado (prova de que o gate funciona)
9 — Databricks	Pipeline (PySpark + dbt) rodando no Community Edition	Execução completa sem erro, com output exportado como evidência
10 — Documentação final	README com arquitetura, decisões e instruções	Alguém de fora consegue reproduzir o pipeline seguindo só o README

Cada fase só é marcada concluída com o critério verificável correspondente cumprido — conforme a regra da Seção 3.2. Dado o escopo ampliado (Seção 16), as fases 4–9 podem ser reordenadas conforme dificuldade encontrada, mas nenhuma é pulada.

16. Riscos, limitações e decisões em aberto

Risco principal: escopo ampliado (decisão da autora em 15/09/2026)

A autora optou explicitamente por um projeto único cobrindo PySpark + Delta Lake + dbt + Airflow + Databricks, em vez de dividir em entregas menores mais rápidas. Isso quase dobra o número de fases (de 7 para 10) e introduz duas ferramentas totalmente novas ao mesmo tempo (Airflow exige um scheduler rodando via Docker; dbt exige um projeto e um profile configurados corretamente). O risco real é abandonar o projeto no meio, com Bronze/Silver prontos mas dbt/Airflow incompletos.

Mitigação adotada:

a Seção 15 mantém cada fase com critério de conclusão verificável e independente — se o tempo apertar, o projeto pode parar ao final da Fase 6 (PySpark + Gold ainda em PySpark, sem dbt/Airflow) e já ser um portfólio funcional, publicável e honesto, em vez de um projeto quebrado tentando fazer tudo.

Outras limitações conhecidas, declaradas com antecedência

- Databricks Community Edition tem limitações de cluster (sem autoscaling, sem suporte a jobs agendados) — suficiente para demonstrar domínio da ferramenta, não para simular produção em escala.
- O ambiente local de desenvolvimento roda Spark em modo standalone (não cluster distribuído de verdade) — isso será declarado com honestidade no README, sem sugerir experiência de cluster multi-nó que não existe.
- Airflow local via Docker Compose não é o mesmo que operar um Airflow gerenciado em produção com múltiplos times e SLAs (ver callout da Seção 9.2).
- O pipeline Bronze/Silver/Gold é validado em profundidade sobre um único mês (202601, 19,48 milhões de linhas). A série histórica (Seção 10) reaproveita o mesmo pipeline, mas em amostragem trimestral (~12 competências) — não é reprocessamento em grão completo de todas as ~35 competências disponíveis (~70 GB), o que fica fora de escopo por limitação de disco, não por decisão de adiar a análise temporal.

17. Conexão com o mercado de trabalho

Mapeamento direto entre o que este projeto entrega e as exigências recorrentes observadas na pesquisa de vagas de setembro/2026:

Exigência recorrente nas vagas	Onde este projeto demonstra
PySpark	Seção 7 — camadas Bronze e Silver em PySpark
Databricks	Fase 9 — pipeline completo rodando no Community Edition
Arquitetura lakehouse/medallion	Seção 6 — bronze/silver/gold explícito, com diagrama
Delta Lake	Seções 6 e 7 — formato de armazenamento nas 3 camadas, com time travel

dbt + SQL avançado	Seção 8 — modelos staging/marts e testes declarativos
Airflow / orquestração	Seção 9 — DAG completa com 5 tasks e dependências
Qualidade de dados / DataOps	Seção 12 — validações PySpark + testes dbt
CI/CD	Seção 12 — GitHub Actions cobrindo pytest e dbt test
Governança e LGPD	Seção 5 — pseudonimização explícita, diferencial raro em projetos de portfólio

18. Referências

- Databricks. *What is the medallion lakehouse architecture?* Documentação oficial. docs.databricks.com/aws/en/lakehouse/medallion
- Controladoria-Geral da União (CGU). *Portal da Transparência — Download de Dados: Novo Bolsa Família*. portaldatransparencia.gov.br/download-de-dados/novo-bolsa-familia
- Delta Lake Project. *Delta Lake Documentation*. delta.io
- dbt Labs. *dbt-databricks adapter documentation*. docs.getdbt.com
- Apache Airflow. *Documentation*. airflow.apache.org/docs
- Astronomer. *Cosmos — run dbt Core projects as Airflow DAGs*. astronomer.github.io/astronomer-cosmos
- Lei nº 13.709/2018 — Lei Geral de Proteção de Dados Pessoais (LGPD)
- Pesquisa interna de vagas ativas (Gupy, Indeed, LinkedIn, GeekHunter) realizada em setembro/2026 como parte do planejamento de carreira da autora